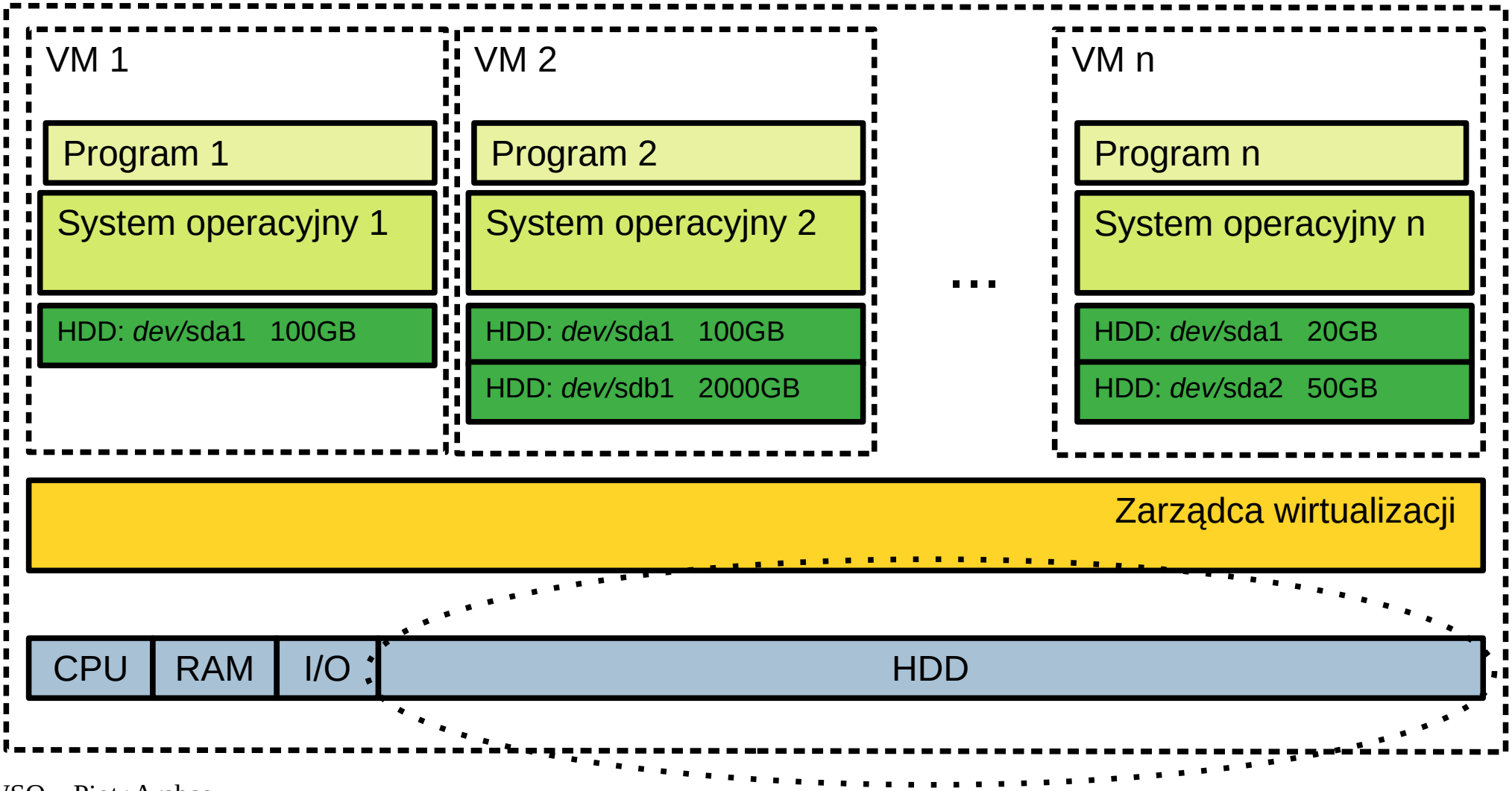
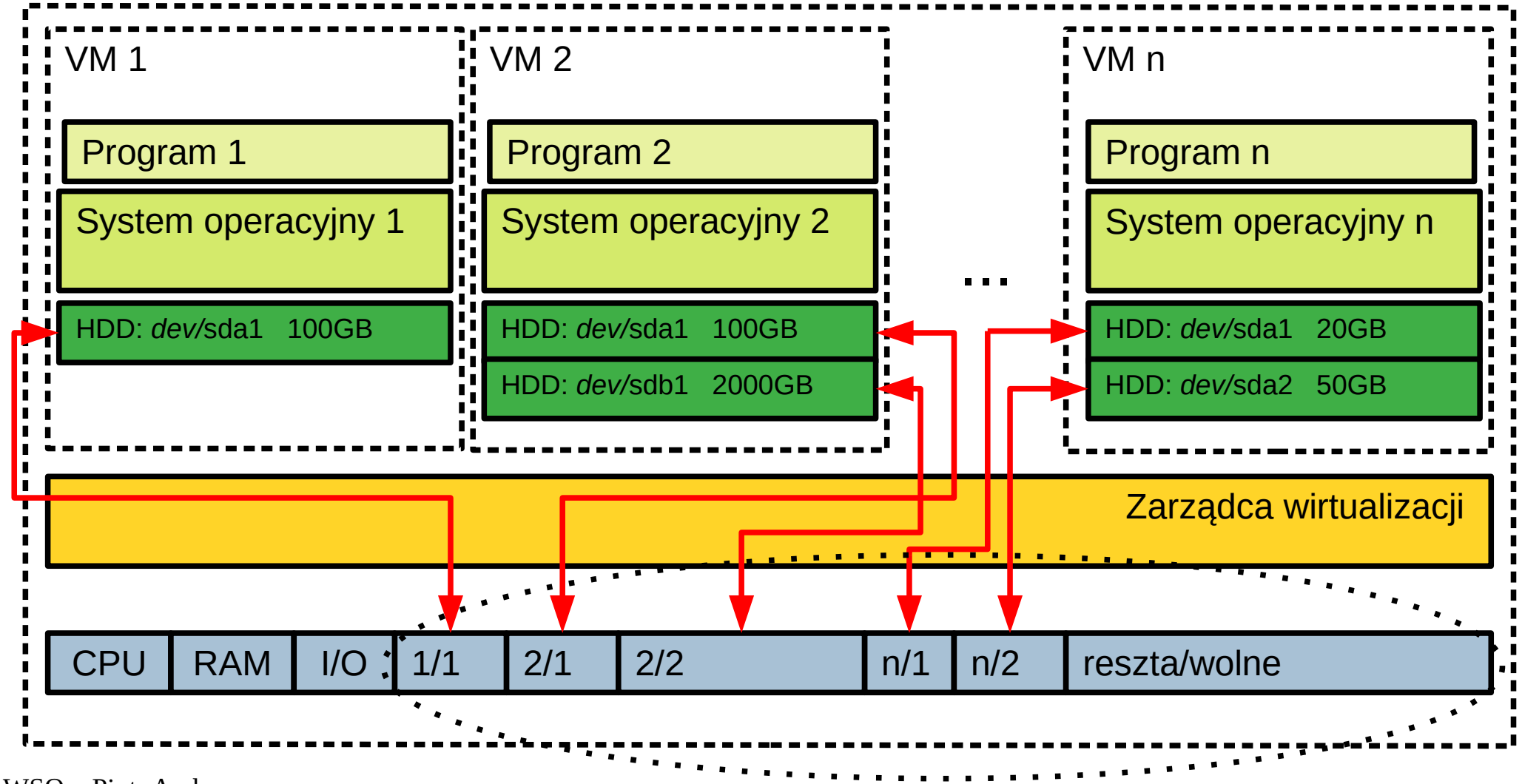
Zarys koncepcji

- Ilustracja: zasoby dyskowe



Zarys koncepcji

- Ilustracja: zasoby dyskowe



Rodzaje wirtualizacji

- Pełna emulacja
- Pełna wirtualizacja
- Wirtualizacja sprzętowa
- Parawirtualizacja
- Wykorzystanie wspólnego jądra

Rodzaje wirtualizacji

- Pełna emulacja
  - wirtualizator przejmuje wszystkie wywołania
  - sprzęt odwzorowany programowo
  - może udawać inny sprzęt
  - przykład: emulator c64 na PC

- Pełna wirtualizacja
- Wirtualizacja sprzętowa
- Parawirtualizacja
- Wykorzystanie wspólnego jądra

Rodzaje wirtualizacji

- Pełna emulacja

- Pełna wirtualizacja
  - Wirtualizator śledzi wywołania
  - Udostępnia zasoby w miarę możliwości bezpośrednio
  - W pozostałych przypadkach emulacja programowa
  - Nie wymaga modyfikacji wirtualizowanego systemu operacyjnego
  - Jeden typ sprzętu
  - Przykłady: VirtualBox, (KVM, Vmware ESXi)

- Wirtualizacja sprzętowa
- Parawirtualizacja
- Wykorzystanie wspólnego jądra

Rodzaje wirtualizacji

- Pełna emulacja
- Pełna wirtualizacja
- Wirtualizacja sprzętowa (pełna wirtualizacja z wparciem sprzętowym)
  - instrukcje „wykonywane bezpośrednio na sprzęcie”
    - procesor wspomaga VMM w wykonaniu uprzywilejowanych instrukcji
  - konieczny procesor z rozszerzonym zestawem instrukcji
  - przykłady: VirtualBox, KVM, Vmware ESXi

Rodzaje wirtualizacji

- Pełna emulacja
- Pełna wirtualizacja
- Wirtualizacja sprzętowa
- Parawirtualizacja
  - Wymagana modyfikacja jądra systemu wirtualizowanego
  - Wirtualizator tworzy warstwę abstrakcji dla części wywołań
  - Przykład: Xen
  - Elementy obecne w większości nowoczesnych wirtualizatorów
    - np. sterowniki virtio w Linuxie dla KVM
- Wykorzystanie wspólnego jądra

Rodzaje wirtualizacji

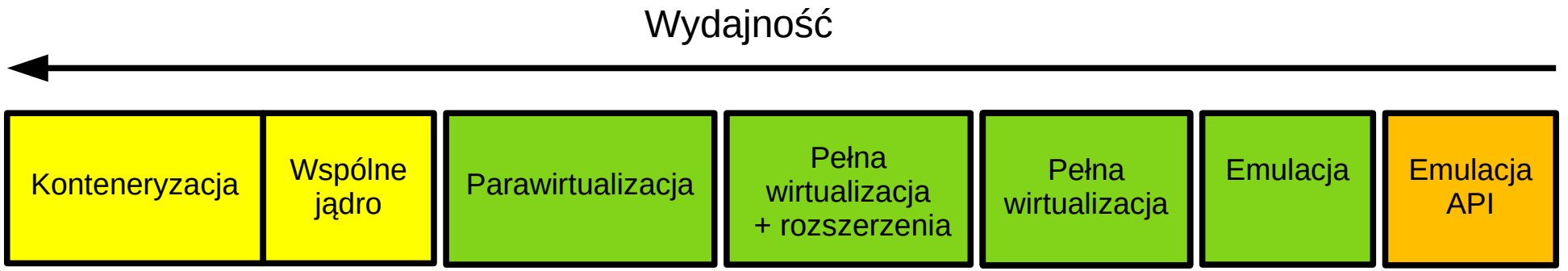
- Pełna emulacja
- Pełna wirtualizacja
- Wirtualizacja sprzętowa
- Parawirtualizacja
- Wykorzystanie wspólnego jądra
  - kilka instancji tego samego systemu
  - OpenVZ (wersja 6)
  - pomysł częściowo wyparty przez konteneryzację

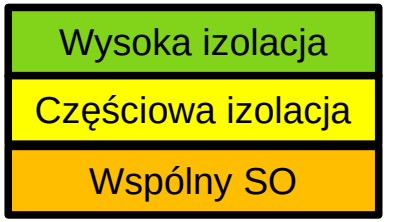
Uwaga: niektóre środowiska klastrowe używają predefiniowanych obrazów systemu w standardowym wirtualizatorze (np. KVM) co może sprawiać (błędne) wrażenie, że maszyny wirtualne mają wspólne jądro. Przykład: Ganeti

Rodzaje wirtualizacji

- Konteneryzacja
  - Wspólne jądro
  - Izolacja procesów i zasobów w kontenerze
  - Znacznie lżejsza niż wirtualizacja
  - Może być stosowana razem z wirtualizacją
  - Przykład: docker
- Emulacja API
  - Pozwala uruchamiać programy z innego systemu operacyjnego
  - Ograniczone możliwości
  - Przykład: wine
- Zagnieżdzona wirtualizacja

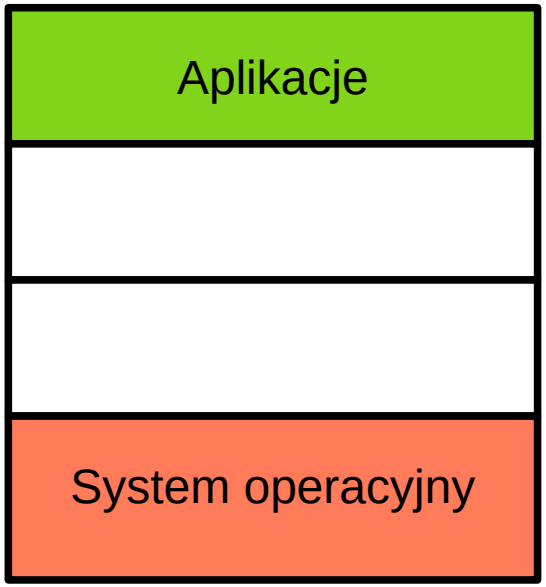
Rodzaje wirtualizacji





- Praktycznie każdy współczesny wirtualizator:
  - Używa rozszerzeń sprzętowych, gdy są dostępne
  - Działa w trybie pełnej wirtualizacji, gdy jest to możliwe
  - Działa w trybie emulacji, gdy jest to konieczne
  - Ma elementy parawirtualizacji

Intel: poziomy uprzywilejowania



Ring 3

Ring 2

Ring 1

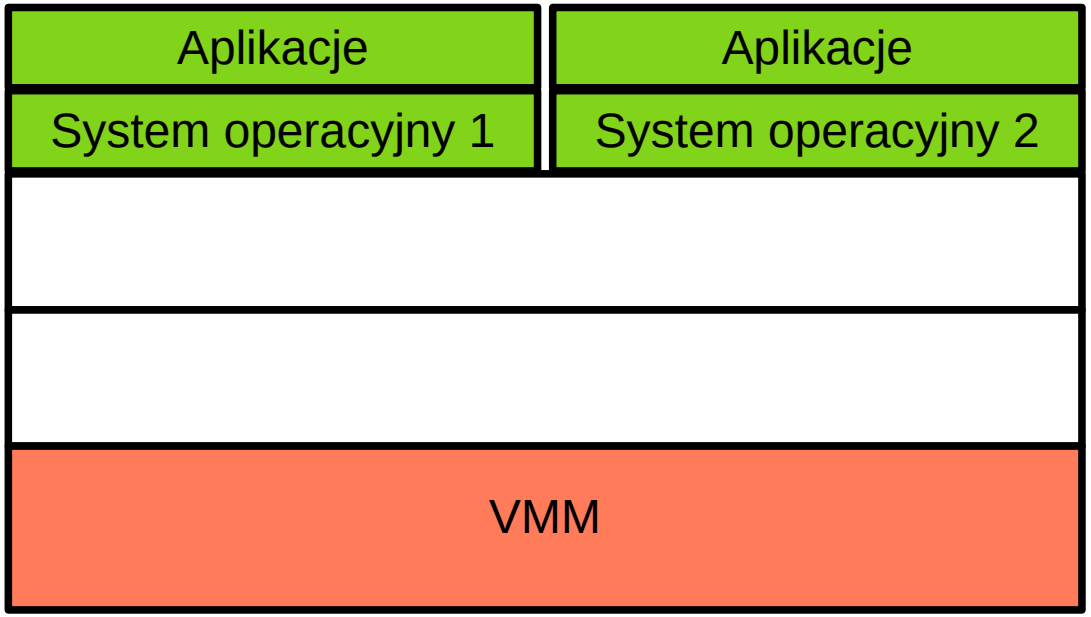
Ring 0

- 4 poziomy uprzywilejowania
- Instrukcje uprzywilejowane np. MOV CR mogą być wykonywane na poziomie 0
- System operacyjny działa na poziomie 0
- Aplikacje działają na poziomie 3
- Wykonanie instrukcji uprzywilejowanej z poziomu innego niż 0 → błąd (*fault*)

Intel: poziomy uprzywilejowania

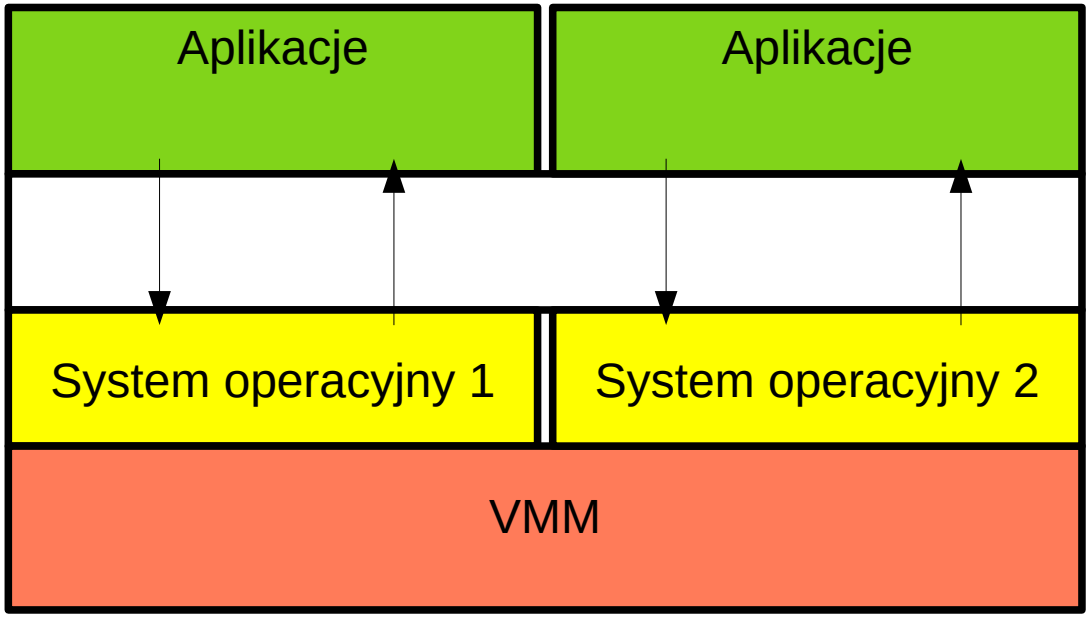
- Uprzywilejowane rejestry
  - np. FLAGS
    - IF – *interrupt flag* – maskuje przerwania, IO – *i/o priviledge level* – zezwala na dostęp do I/O
    - Może być załadowany instrukcją POPF
  - Rejestry CR1-3, DR0-7, IO (rejestry urządzeń)
- Uprzywilejowane instrukcje, np.:
  - HLT
  - LIDT – *load interrupt descriptor table*
  - RDMSR – *read model specific registers*
- Wywołanie instrukcji uprzywilejowanej bez uprawnień powoduje *general protection fault*
  - Wyjątek: POPF – brak błędu

Intel: poziomy uprzywilejowania



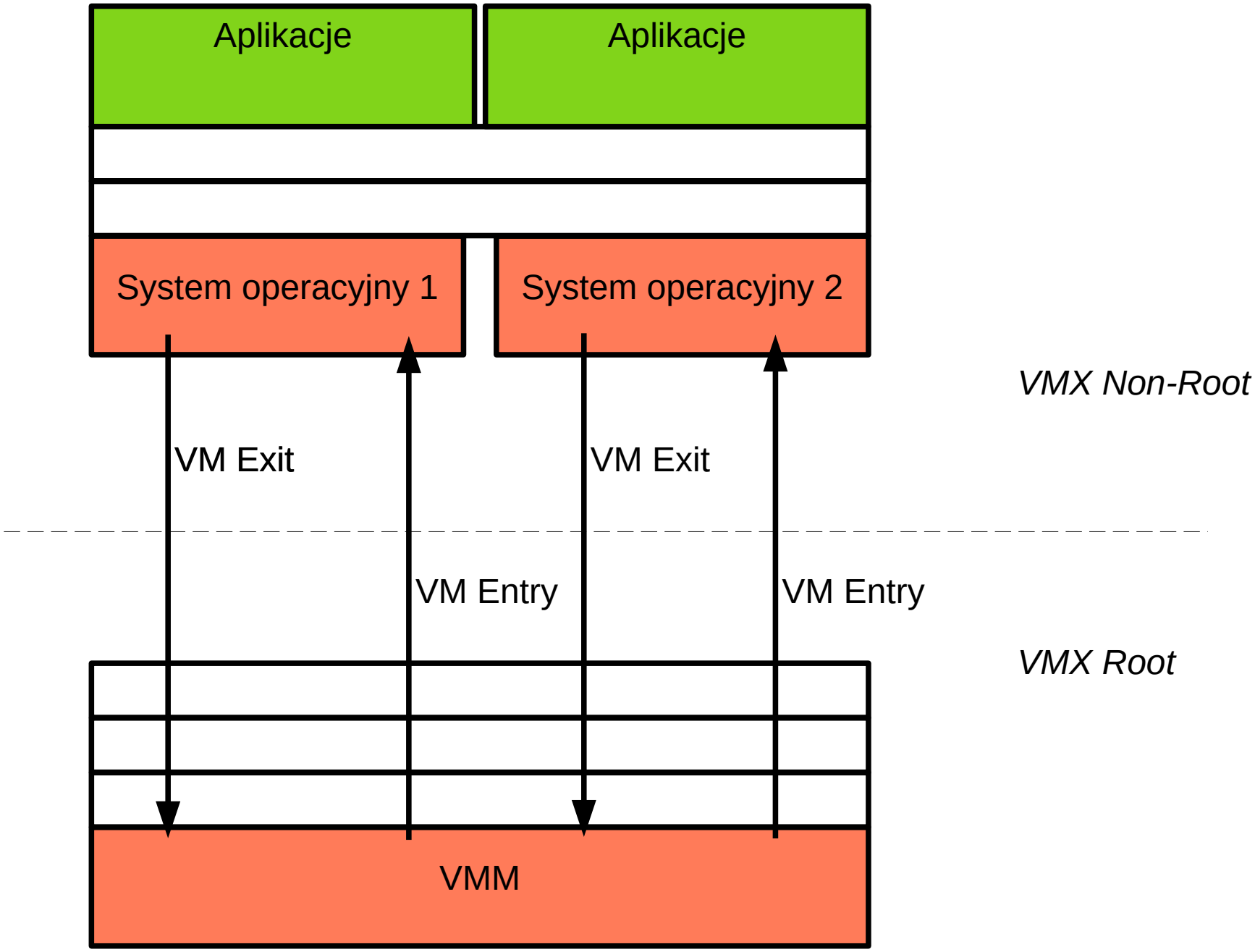
- Model 0/3/3
- Instrukcje uprzywilejowane → błąd (*fault*) → VMM → emulacja
- Niewydajne
- Potencjalnie niebezpieczne
- System operacyjny działa na niewłaściwym poziomie
- *Ring aliasing / address space compression*

Intel: poziomy uprzywilejowania



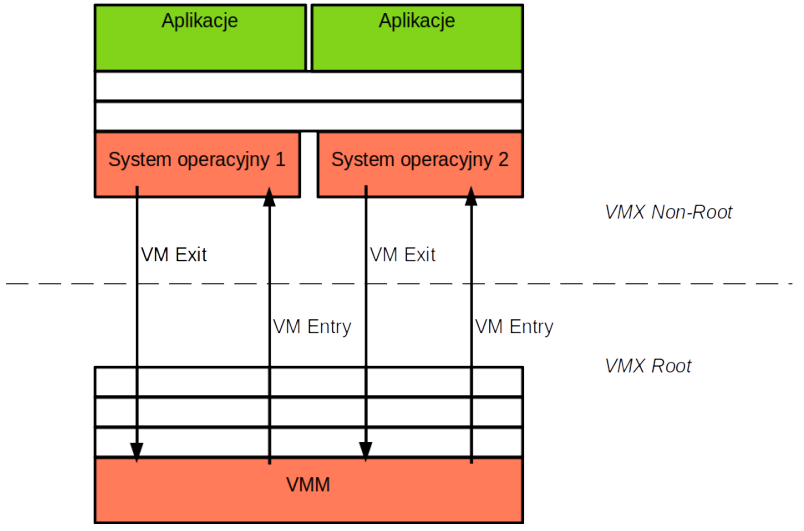
- Model 0/1/3
- Instrukcje uprzywilejowane → błąd (*fault*) → VMM → emulacja
- Niewydajne
- Potencjalnie niebezpieczne
- System operacyjny działa na niewłaściwym poziomie
- *Ring aliasing / address space compression*

Intel: rozszerzenia VT-x



Intel: rozszerzenia VT-x

- System operacyjny → ring 0
- VMM → ring 0
- VMCS (*Virtual Monitor Control Structure*) – zawiera „powód wyjścia”
  - każdy vCPU ma swój VMCS
- Pojedynczy punkt wejścia do VMM
- Procesor dba o przekazanie kontekstu i ochronę pamięci
- Obsługa wejścia-wyjścia – VT-d
  - PCIe-passthrough



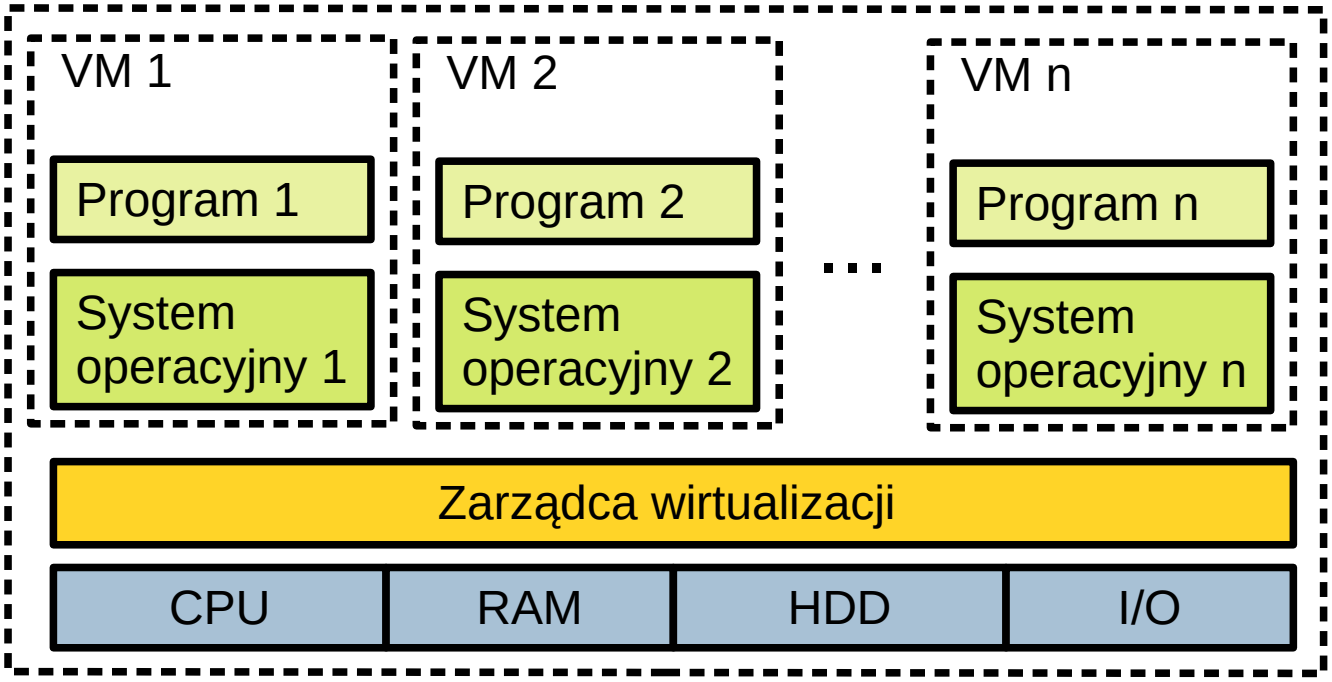
Intel: rozszerzenia VT-x

- Jak sprawdzić?
  - `grep vmx /proc/cpuinfo`
- A co z AMD?
  - AMD ma instrukcje AMD-V
  - `grep svm /proc/cpuinfo`

- Również `lscpu`

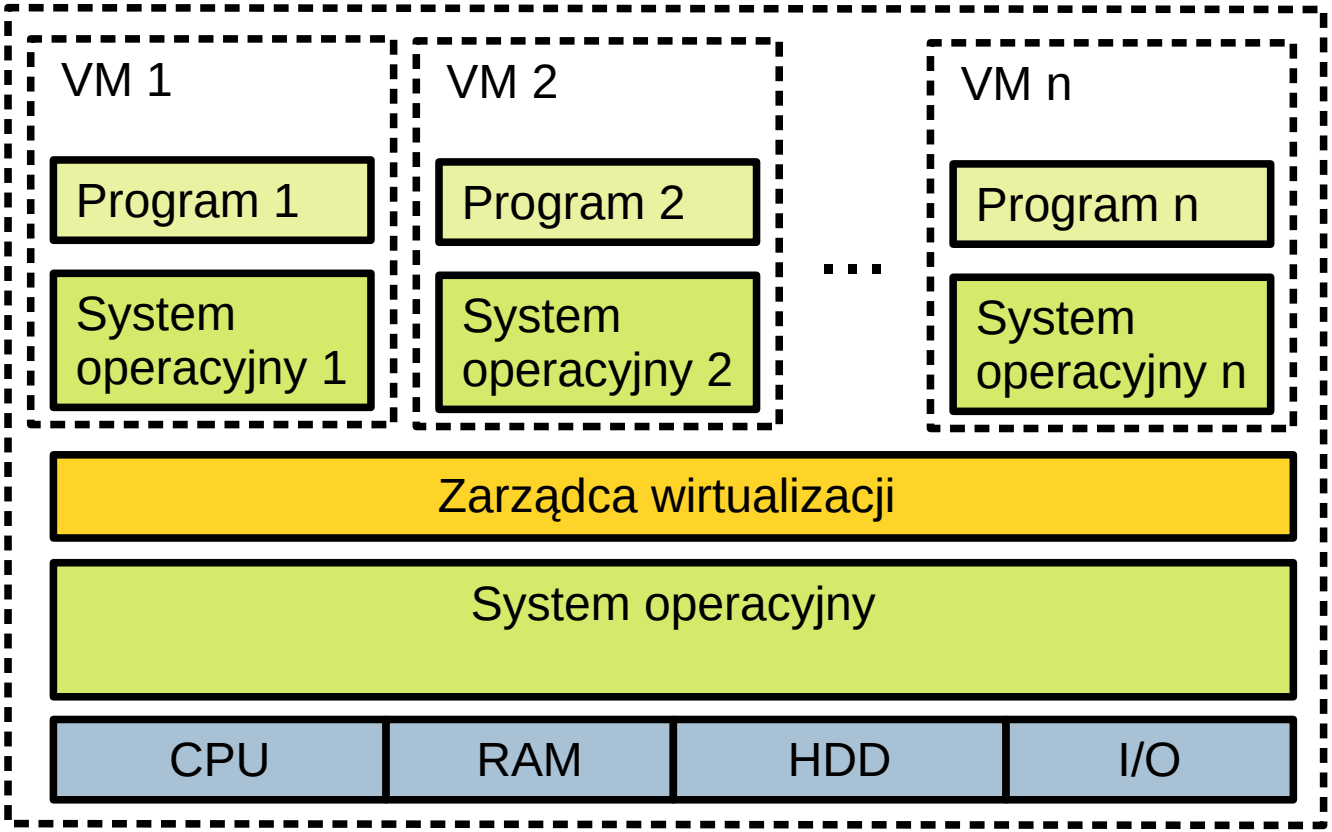
Rodzaje wirtualizacji

- Zarządca wirtualizacji typu 1
  - Działa bezpośrednio na maszynie fizycznej
  - *Bare metal*



Rodzaje wirtualizacji

- Zarządca wirtualizacji typu 2
  - Wymaga systemu operacyjnego gospodarza



Rodzaje wirtualizacji

- Zarządca wirtualizacji typu 1
  - Przykład: Vmware ESXi
  - Działa bezpośrednio na sprzęcie
    - Wydajniejszy
    - Bezpieczniejszy
- Zarządca wirtualizacji typu 2
  - Przykład: VirtualBox
  - Wymaga systemu operacyjnego gospodarza
    - Należy dbać o bezpieczeństwo (też aktualizacje!) obu
    - Może być mniej wydajny
    - Ale jest łatwy do doinstalowania

Rodzaje wirtualizacji

- Uwagi, kontrowersje
  - Typ 2 pozwala na prostszy sposób wdrożenia
  - KVM jest modułem jądra
    - twórcy: jest typu 1 bo obsługuje sprzęt
    - przeciętny użytkownik: ale musiałem zainstalować system ???
  - Zalety dojrzałych komercyjnych wirtualizatorów są często utożsamiane z zaletami typu 1
    - np. łatwa aktualizacja Vmware ESXi
    - bezpieczeństwo j.w.
  - Używając typu 1 unikamy kosztu systemu operacyjnego. Pytanie: ile kosztuje wirtualizator?